

SkillNova Technical Implementation Report

What we built, how it works, where every important file lives, and how to run, test, explain, and extend the MVP.



Project	SkillNova AI Talent Intelligence
Repository	github.com/krazyDarshan/logic_loop
Reviewed commit	0fc069d - fresh-clone production build verified
Report date	29 July 2026
Deployment	skillnova-talent-intelligence.darshanbawaskar.chatgpt.site

Decision support only. Human review remains required for every hiring decision.

00 / CONTENTS AND REPORT SCOPE

Contents and report scope

Contents and report scope	2
Executive summary	3
Scope decisions and delivery history	4
System architecture	5
Core data model and sample talent pool	6
Resume and GitHub intelligence	7
Talent Score calculation	8
Job matching engine	9
Skill verification and interview agent	10
Hackathon-to-hiring pipeline	11
Recruiter workspace and Trust Center	12
User interface and experience design	13
File-by-file implementation guide	14
Important code paths	16
Installation and operating commands	17
Testing and deployment	18
Current limitations and production risks	19
Recommended roadmap	20
Demo and handoff guide	21

Scope statement

This document describes the code that exists in the repository today. It separates real calculations and live data access from seeded demo data, UI-only controls, and future production work.

How to use this report

- Use Sections 1-4 to explain the product and architecture to judges or teammates.
- Use Sections 5-10 to understand the algorithms and module implementations.
- Use Sections 11-13 as the file-by-file developer reference.
- Use Sections 14-17 for setup, tests, deployment, risks, and roadmap.

01 / EXECUTIVE SUMMARY

Executive summary

SkillNova is a client-first, evidence-driven recruitment MVP. It converts a resume, a public GitHub profile, structured hackathon evidence, and interview responses into an explainable candidate profile. The main product principle is proof over paperwork: every visible score is derived from published rules instead of an opaque model response.

What the finished application includes

- A candidate intelligence workspace that extracts resume text and retrieves live public GitHub signals.
- A seven-dimension Talent Score with confidence and evidence-count indicators.
- A job matching engine that reacts immediately when the job description changes.
- An adaptive interview flow and a browser-based executable coding challenge.
- A hackathon leaderboard that turns project evidence into recruiter-ready profiles.
- A recruiter workspace with natural-language filtering, comparison, shortlisting, and PDF export.
- A Trust Center with authenticity calculations, fraud indicators, safeguards, and an audit view.
- Ten seeded candidate profiles so every module can be demonstrated without external setup.

What 'AI' means in this MVP

The implemented intelligence is deterministic and inspectable. Resume extraction, taxonomy matching, GitHub signal analysis, weighted scoring, adaptive question generation, answer evaluation, and natural-language filters all run from explicit code. No OpenAI or Llama API is currently called, and no trained prediction model is deployed.

Final technology stack

Layer	Implemented technology	Purpose
Interface	Next.js 16, React 19, TypeScript	Single-page recruiter and candidate workspace
Styling	Custom CSS and Tailwind PostCSS plugin	Soft-light visual system and responsive layouts
Build	Vinext, Vite 8	Compile the Next app for Cloudflare-compatible runtime
Runtime	Cloudflare Worker and Wrangler	Server rendering, assets, and local production simulation
Resume	PDF.js	Extract text from up to 20 PDF pages in the browser
GitHub	Public GitHub REST API	Repositories, events, languages, stars, and repository contents
Reports	jsPDF	Generate recruiter-ready candidate reports in the browser
Database scaffold	Drizzle ORM and optional D1	Prepared but not active in the current MVP
Hosting	OpenAI Sites	Private production deployment of the verified Vinext bundle

02 / SCOPE DECISIONS AND DELIVERY HISTORY

Scope decisions and delivery history

The original challenge contained more than twelve mandatory deliverables. The project was intentionally narrowed to five core MVP pillars, then a trust module and operational polish were added. This kept the demo coherent while still covering candidate discovery, verification, evaluation, and recruiter action.

MVP scope mapping

Requested MVP pillar	Implemented workspace	Status
AI Resume and GitHub Analysis	Talent Intelligence	Implemented with real local extraction and live public API calls
AI Job Matching Engine	Job Matching	Implemented with transparent weighted scoring
Skill Verification and Interview Agent	Skill Verification	Implemented with adaptive prompts, rubric scoring, and code tests
Hackathon-to-Hiring Pipeline	Hackathon Hiring	Implemented with calculated project ranking and recruiter handoff
Recruiter Dashboard	Recruiter Workspace	Implemented with search, comparison, shortlist, and PDF export
Trust and Fraud Prevention	Trust Center	Implemented as formula-driven indicators and governance UI

Implementation timeline in Git

Commit	Milestone
c07f383	Built the initial proven talent intelligence platform
b1529b5	Focused the product on the SkillNova MVP
24268a5	Refined the enterprise visual system
e0ad768	Moved the interface to the softer light theme
1d24570	Completed the evidence engine, expanded dataset, tests, and final workflows
0fc069d	Removed the README; application behavior was unchanged

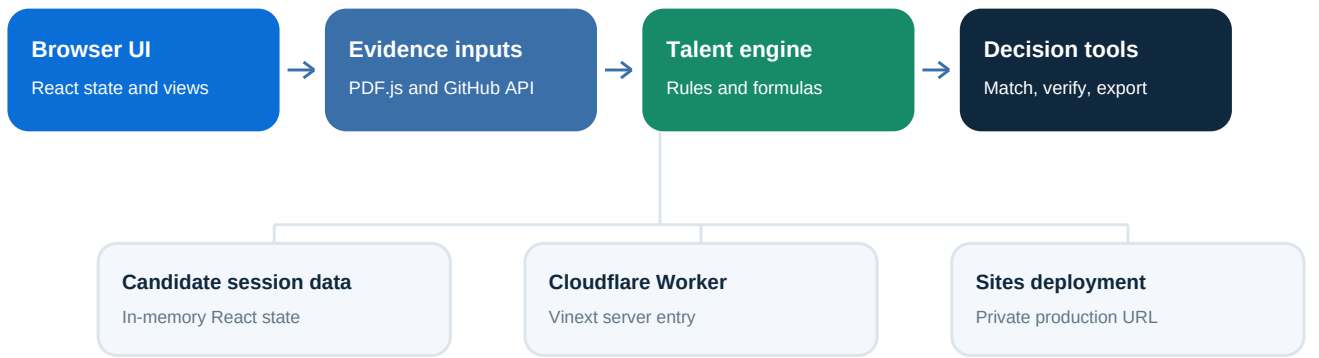
Features deliberately not present

The full challenge modules for presentation analysis, career guidance, ATS resume generation, voice/video interviewing, salary prediction, and a persistent recruiter pipeline were removed from the active MVP. This report does not claim them as implemented.

03 / SYSTEM ARCHITECTURE

System architecture

The application uses a browser-first architecture. React holds the current candidate pool, selection, interview state, shortlist, safeguards, and comparison list. External evidence is read directly in the browser, normalized by the talent engine, then rendered across six workspaces. The compiled Next.js application is served through a Cloudflare Worker-compatible Vinext bundle.



Key architecture properties

- No separate FastAPI service is required for the current demo.
- Resume contents remain in the active browser session and are not uploaded to a project database.
- GitHub evidence comes from unauthenticated public REST endpoints.
- The talent, matching, authenticity, search, and interview formulas are shared TypeScript logic.
- A page refresh resets live candidate additions, shortlist state, controls, and interview progress.
- The database and authentication helpers are scaffolding for a future persisted version.

Primary data flow

Step	Operation	Result
1	Choose a PDF or text resume	Browser receives a File object
2	Extract text with PDF.js	Plain text for taxonomy and experience parsing
3	Enter a public GitHub username	User, repo, event, and top-repo content responses
4	Normalize evidence	Skills, languages, repository quality, activity, and trust inputs
5	Calculate Talent Score	Seven dimensions, total, confidence, and evidence count
6	Propagate the candidate	Matching, verification, hackathon, recruiter, and trust views update
7	Recruiter acts	Compare, shortlist, interview, and download a hiring report

04 / CORE DATA MODEL AND SAMPLE TALENT POOL

Core data model and sample talent pool

The data model lives in app/talent-engine.ts. CandidateBase stores the raw evidence and Candidate adds the calculated TalentScore. This separation makes recalculation predictable whenever live resume or GitHub evidence replaces the sample values.

Important TypeScript structures

Type	Important fields	Role
SkillSignal	name, level, verified, sources	One normalized capability and its provenance
CandidateBase	identity, role, location, experience, skills	Candidate identity and normalized evidence
github	repos, stars, commits, pullRequests, activeWeeks, languages	Repository activity and quality inputs
resume	education, projects, certifications	Structured resume-derived inputs
hackathons	count, wins, innovation, project	Project and innovation evidence
interview	technical, communication, problemSolving	Verification inputs
trust	identity, resumeConsistency, repoAuthenticity, duplicateRisk	Authenticity inputs
TalentDimensions	coding, projectQuality, problemSolving, consistency, leadership, innovation, community	Seven scored dimensions
JobMatch	total, four factors, matched and missing skills	Explainable role fit result

Seeded candidate dataset

Candidate	Role	Location	Representative skills
Ananya Verma	Full-stack AI Engineer	Bengaluru	Python, React, FastAPI, LangGraph
Rohan Mehta	Machine Learning Engineer	Delhi	Python, PyTorch, MLOps, Docker
Meera Nair	Frontend Engineer	Pune	React, TypeScript, Next.js, GraphQL
Kabir Shah	Backend Engineer	Mumbai	FastAPI, PostgreSQL, Redis, AWS
Ishita Rao	Generative AI Engineer	Hyderabad	GenAI, RAG, LangGraph, Python
Arjun Singh	Cloud and DevOps Engineer	Gurugram	AWS, Kubernetes, Docker, CI/CD
Neha Kulkarni	Data Engineer	Chennai	SQL, Spark, Python, AWS
Vikram Das	Mobile Product Engineer	Kolkata	Flutter, Kotlin, React Native
Sara Khan	Security Engineer	Bengaluru	Cybersecurity, Linux, Python, CI/CD
Dev Patel	Product Engineer	Ahmedabad	TypeScript, React, Next.js, Node.js

Demo data label

These ten profiles are seeded examples for a complete demonstration. Their score inputs are authored sample data. A live analyzed GitHub profile is inserted at the top of the in-memory pool and is clearly labeled Live API.

05 / RESUME AND GITHUB INTELLIGENCE

Resume and GitHub intelligence

Resume extraction

- extractFileText accepts PDF and text files from the browser file picker.
- PDF.js is dynamically imported so its heavy bundle is loaded only when needed.
- The PDF worker reads a maximum of 20 pages and concatenates text items page by page.
- parseResumeText matches a catalog of approximately thirty normalized technologies and aliases.
- Experience comes from year patterns, education is categorized at a basic level, and project evidence is estimated from action keywords.

```
async function extractFileText(file) {
  const pdfjs = await import('pdfjs-dist/legacy/build/pdf.mjs');
  const pdf = await pdfjs.getDocument({ data: bytes }).promise;
  for (let page = 1; page <= Math.min(pdf.numPages, 20); page++) {
    // Read text items and join them into resume text.
  }
}
```

GitHub evidence retrieval

Endpoint or analysis	What is collected
/users/{username}	Login and public display name
/users/{username}/repos	Up to 100 repositories sorted by update time
/users/{username}/events/public	Up to 100 recent public events
/repos/{owner}/{repo}/contents	Top-level files for the five highest-priority original repositories
Repository filtering	Forks are removed before original-project scoring
Language analysis	Six most frequent primary repository languages
Quality signals	Test folders, README files, descriptions, stars, and originality ratio

GitHub heuristic formulas

```
testing      = clamp(48 + testRepoRatio * 48)
documentation = clamp(45 + descriptionRatio * 28 + readmeRatio * 25)
originality  = clamp(70 + originalRepoRatio * 28)
activeWeeks  = min(44, max(activeMonths * 4, originalRepos * 2))
commitSignal = max(recentPushPayloadSize, originalRepos * 6)
prSignal     = max(recentPublicPREvents, round(originalRepos * 0.8))
```

Important accuracy boundary

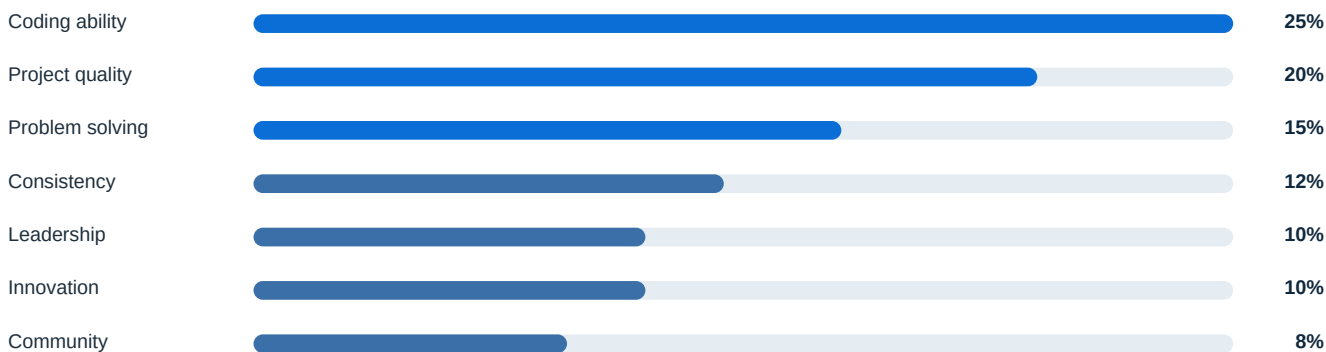
GitHub public events are a recent window, not a candidate's lifetime contribution history. The fallback multipliers keep the demo usable but should not be presented as verified lifetime commit or pull-request counts.

06 / TALENT SCORE CALCULATION

Talent Score calculation

calculateTalentScore converts mixed evidence into comparable 0-100 dimensions. Input scales cap repository counts, commits, stars, pull requests, projects, and active weeks so one unusually large number cannot dominate the result. Every value is rounded and clamped between 0 and 100.

Overall weighting



```
Talent Score = Coding x 0.25 + Project Quality x 0.20
               + Problem Solving x 0.15 + Consistency x 0.12
               + Leadership x 0.10 + Innovation x 0.10
               + Community x 0.08
```

Dimension formulas

Dimension	Formula	Interpretation
Coding	$\text{skillAvg} \times .42 + \text{repoScale} \times .16 + \text{commitScale} \times .17 + \text{tests} \times .10 + \text{originality} \times .15$	Technical breadth, activity, testing, and original work
Project quality	$\text{documentation} \times .22 + \text{tests} \times .20 + \text{starScale} \times .12 + \text{originality} \times .24 + \text{projectScale} \times .22$	Engineering hygiene, public validation, and project depth
Problem solving	$\text{skillAvg} \times .30 + \text{interviewProblemSolving} \times .45 + \text{hackathonInnovation} \times .25$	Skills plus demonstrated reasoning and innovation
Consistency	$\text{activeScale} \times .45 + \text{resumeConsistency} \times .35 + \text{verifiedRatio} \times 100 \times .20$	Sustained activity and agreement between claims and evidence
Leadership	$\text{hackathonParticipationIndex} \times .55 + \text{communication} \times .25 + \text{prScale} \times .20$	Winning/participation, communication, and collaborative contribution
Innovation	$\text{hackathonInnovation} \times .58 + \text{originality} \times .27 + \text{projectScale} \times .15$	Idea novelty backed by original implementation
Community	$\text{prScale} \times .46 + \text{starScale} \times .22 + \text{hackathonCountScale} \times .18 + \text{documentation} \times .14$	Collaboration, visibility, participation, and reusable documentation

Confidence and evidence volume

```
evidenceCount = repos + resumeProjects + pullRequests
                + hackathonCount + verifiedSkillCount

confidence = clamp(55 + min(evidenceCount, 35) x 0.9
                  + resumeConsistency x 0.1)
```

Why the formula is judge-friendly

A recruiter can see the input categories, weights, evidence count, and source labels. The model is easier to explain, challenge, and recalibrate than a single untraceable LLM score.

07 / JOB MATCHING ENGINE

Job matching engine

The Job Matching workspace continuously recalculates every candidate against the editable job description. `extractRequiredSkills` applies the same skill taxonomy used for resumes, which avoids separate naming systems for candidates and roles.

Match formula

```
Job Match = Skill Similarity x 0.45
            + Project Relevance x 0.25
            + Experience Fit x 0.20
            + Evidence Confidence x 0.10
```

Factor	Calculation	Weight
Skill similarity	72% skill coverage plus 28% depth of matched skills	45%
Project relevance	Keyword overlap with summary, project, languages, and skills plus project-quality contribution	25%
Experience fit	Candidate years divided by parsed required years, with a bonus for meeting the requirement	20%
Evidence confidence	72% Talent Score confidence plus 28% resume consistency	10%

Output and recruiter behavior

- The selected candidate receives a total match percentage and a text explanation.
- Matched requirements and missing or unverified skills are shown separately.
- The entire pool is sorted by match score and the top six candidates are displayed.
- Changing one technology or year requirement triggers immediate recalculation through `useMemo`.
- The interface labels 90+ as exceptional, 80-89 as strong, 70-79 as promising, and lower results as partial fit.

Semantic search boundary

The current engine is taxonomy and token-overlap based. It does not create vector embeddings or call `pgvector`. For the hackathon MVP this makes the score fast and explainable, but semantic synonyms outside the catalog may be missed.

08 / SKILL VERIFICATION AND INTERVIEW AGENT

Skill verification and interview agent

Adaptive interview

makeQuestions selects the candidate's three highest-priority skills and produces three targeted prompts: a production decision, a reliable service design, and an incident investigation. Each question includes a short rubric of observable concepts rather than evaluating voice, facial expression, or personality.

Answer scoring

```
coverage = matchedRubricTerms / rubricTermCount
depth    = min(answerWordCount / 120, 1)
structure = min(reasoningConnectorCount / 3, 1)

answerScore = clamp(45 + coverage x 32
                    + depth x 15 + structure x 8)
```

Question type	Representative rubric terms
Production decision	trade-off, test, production, monitor
System design	failure, scale, test, observability
Incident response	reproduce, logs, communication, rollback
Reasoning structure	because, first, then, however, therefore

Executable coding assessment

- The candidate edits a JavaScript normalizeSkills function.
- A Blob-based Web Worker evaluates the submitted function away from the main UI thread.
- Three tests check trimming, lowercasing, deduplication, alphabetical sorting, and the empty-array case.
- The worker terminates after two seconds to prevent a hanging assessment.
- The UI displays passed tests and a targeted remediation message.

Production security note

A Web Worker with eval is useful for a controlled demo but is not a hardened code sandbox. A production release should submit code to Judge0 or an isolated container with CPU, memory, network, and filesystem controls.

09 / HACKATHON-TO-HIRING PIPELINE

Hackathon-to-hiring pipeline

HackathonView filters candidates with participation, sorts them by innovation, and presents event-level statistics plus a recruiter-ready project profile. This reuses the same evidence model instead of creating an unrelated hackathon score.

Project evaluation shown in the interface

Displayed project factor	Source in the candidate model
Innovation	hackathons.innovation
Technical depth	Talent Score coding dimension
Feasibility	Talent Score projectQuality dimension
Authenticity	Calculated authenticity score
Verified technology	Top skills and verification flags
Contribution signal	Pull requests and active contribution weeks

Recruiter handoff

- View talent profile moves the selected finalist into the evidence workspace.
- Invite to verification opens the adaptive interview for the same candidate.
- The event summary counts analyzed projects, recruiter-ready profiles, top innovation, and wins.
- The ranking is reproducible because it sorts the stored innovation values directly.

Current boundary

There is no hackathon import API, presentation/PDF analyzer, team commit attribution, or persistent recruiter invitation workflow in this narrowed MVP. The screen demonstrates the pipeline using the shared candidate dataset.

10 / RECRUITER WORKSPACE AND TRUST CENTER

Recruiter workspace and Trust Center

Recruiter discovery

RecruiterView operates on the full candidate pool. searchCandidates parses the query for catalog skills, known locations, hackathon intent, open-source intent, and a minimum Talent Score. Results are filtered and sorted by Talent Score.

Query intent	Implementation rule
Skill	All recognized requested skills must exist on the candidate profile
Location	A location already present in the pool must match
Hackathon or winner	Candidate must have at least one hackathon
Open source or GitHub	Candidate must have at least 15 pull-request signals
Talent score above N	Candidate total must meet the parsed two-digit threshold
Unstructured text	Fallback substring search over identity, role, location, summary, and skills

Recruiter actions

- Shortlist and remove candidates in local React state.
- Select up to four candidates for side-by-side comparison.
- Compare Talent Score, confidence, coding, projects, problem solving, interview, and authenticity.
- Export a candidate PDF containing score, role match, skills, dimensions, gaps, and recommendation.

Authenticity and governance

```
Authenticity = Identity Confidence x 0.28
              + Resume Consistency x 0.30
              + Repository Authenticity x 0.32
              + (100 - Duplicate Risk) x 0.10
```

- Fraud indicators display resume-GitHub consistency, project originality, duplicate safety, and evidence freshness.
- Safeguard switches cover human review, PII redaction, audit logging, and protected-trait exclusion.
- The audit view lists score calculation, authenticity checks, protected-trait exclusion, and pending human review.

Governance implementation boundary

The safeguard switches and audit timeline are session UI state. They do not yet enforce server-side policies, redact PDF text, write immutable audit records, or run biometric/identity verification.

11 / USER INTERFACE AND EXPERIENCE DESIGN

User interface and experience design

The final visual direction is a soft-light enterprise dashboard: warm off-white application surfaces, white content cards, muted blue accents, dark readable type, teal verification states, and a restrained navy navigation rail. This replaced an earlier overly dark presentation.

Design tokens

Token	Value	Usage
--navy	#061D30	Sidebar, dark emphasis, and high-contrast surfaces
--cobalt	#0B6ED7	Primary actions, scores, links, and progress
--ink	#10283D	Primary text
--muted	#6C7D8D	Secondary information
--line	#DCE5EB	Borders and separators
--paper	#F7F9FA	Application background
--green	#178B68	Verified and success states
--amber	#E4A128	Review and missing-skill states
--red	#C96055	Errors and risk states

Responsive behavior

Breakpoint	Behavior
Below 1180px	Product hero simplifies, comparison becomes two columns, verification and trust become one column
Below 1100px	Major workspaces collapse to a single column; large decorative evidence visuals hide
Below 800px	Sidebar becomes a slide-over menu; padding reduces; analysis, matching, and copilot layouts stack
Below 540px	Forms and actions stack; stat grids become one column; prompt chips hide; table controls simplify

Reusable interface primitives

- Icon renders the shared inline SVG icon vocabulary.
- ScoreRing uses a conic gradient driven by a CSS custom property.
- SkillPill standardizes neutral, verified, matched, and missing states.
- CandidateSelect keeps the chosen candidate synchronized across workspaces.
- EvidenceExplorer links a skill claim to its resume or GitHub source.
- Content cards, formula boxes, progress bars, status badges, and responsive grids create visual consistency.

12 / FILE-BY-FILE IMPLEMENTATION GUIDE

File-by-file implementation guide

Core application files

File	Responsibility
app/page.tsx	Main client application. Contains the six workspaces, PDF extraction, GitHub calls, interview evaluation, Web Worker assessment, recruiter PDF export, navigation, and React state.
app/talent-engine.ts	Types, skill taxonomy, ten candidate records, Talent Score, resume parsing, job matching, live-candidate merge, authenticity, and recruiter search.
app/globals.css	Complete design system: tokens, sidebar, cards, forms, tables, score rings, module layouts, states, animation, and responsive breakpoints.
app/layout.tsx	Root fonts, metadata, social sharing tags, dynamic origin handling, and Open Graph image configuration.
app/chatgpt-auth.ts	Reusable ChatGPT-hosted sign-in/out header helpers. Present as scaffolding but not called by the current page.
public/og.png	Light-mode SkillNova social preview generated to match the final visual direction.

Build, runtime, and data files

File	Responsibility
package.json	npm scripts, Node >=22.13.0 requirement, application dependencies, and build tooling.
package-lock.json	Pinned npm dependency graph for repeatable npm ci installations.
vite.config.ts	Vinext, Sites, and Cloudflare plugins; local Worker bindings; Wrangler state paths.
worker/index.ts	Cloudflare Worker entry; routes image optimization and all other traffic to the Vinext app handler.
build/sites-vite-plugin.ts	Copies hosting metadata and optional Drizzle migrations into dist/.openai after builds.
next.config.ts	Minimal Next.js configuration placeholder.
tsconfig.json	Strict TypeScript, bundler resolution, React JSX, and @/* path alias.
postcss.config.mjs	Registers the Tailwind CSS PostCSS plugin.
eslint.config.mjs	Next core web vitals and TypeScript ESLint presets.
db/schema.ts	Intentionally empty database schema for the current no-persistence MVP.
db/index.ts	Optional Drizzle D1 connector; throws a clear error if DB is not bound.
drizzle.config.ts	Drizzle migration configuration pointing at db/schema.ts.
drizzle/meta/_journal.json	Migration metadata scaffold.
.openai/hosting.json	Sites project binding configuration; D1 and R2 are currently null.

Quality, examples, and repository control

File	Responsibility
tests/rendered-html.test.mjs	Imports the built Worker, verifies HTTP 200 HTML, checks core UI text, and asserts that real PDF/GitHub/scoring/export code is present.
examples/d1/app/api/notes/route.ts	Optional example API route for using D1 persistence later.
examples/d1/db/schema.ts	Optional example database schema.
.gitignore	Excludes dependencies, generated builds, local caches, environment files, coverage, logs, and editor artifacts.

Repository tree

```
logic_loop/  
  app/  
    page.tsx           # all interactive workspaces  
    talent-engine.ts   # evidence and scoring logic  
    globals.css        # visual system  
    layout.tsx        # metadata and fonts  
    chatgpt-auth.ts    # optional auth helper  
  public/og.png        # social preview  
  tests/rendered-html.test.mjs  
  worker/index.ts      # Cloudflare runtime entry  
  build/sites-vite-plugin.ts  
  db/                  # optional D1 scaffold  
  examples/d1/         # persistence example  
  package.json / package-lock.json  
  vite.config.ts / next.config.ts / tsconfig.json
```

Why some local folders are absent from GitHub

node_modules is recreated by npm ci. .vinext and dist are generated by npm run build. .wrangler is local runtime state. .next, coverage, environment files, and debug logs are also intentionally ignored. A fresh clone was built successfully from GitHub at commit 0fc069d.

13 / IMPORTANT CODE PATHS

Important code paths

Live analysis orchestration

```
const analyze = async () => {
  const resume = file
    ? parseResumeText(await extractFileText(file))
    : null;
  const live = await fetchGithubEvidence(username.trim());
  const merged = mergeLiveCandidate(candidate, resume, live.github, live.name);
  onCandidateAnalyzed(merged);
};
```

Shared candidate state

```
const [pool, setPool] = useState(seededCandidates);
const [selectedId, setSelectedId] = useState(seededCandidates[0].id);

const handleAnalyzed = (candidate) => {
  setPool(current => [candidate, ...current.filter(notPreviousLive)]);
  setSelectedId(candidate.id);
};
```

Hiring report export

- jsPDF is dynamically imported only when the recruiter clicks Download hiring report.
- The generated file contains candidate identity, Talent Score, role match, authenticity, verified skills, dimensions, matched requirements, gaps, and a recommendation.
- The report footer explicitly states that SkillNova is decision support and does not make autonomous hiring decisions.
- The download filename is normalized from the candidate name.

Social metadata

- generateMetadata reads forwarded host and protocol headers so social URLs work locally and after deployment.
- Open Graph and Twitter cards reference public/og.png at 1732 x 908 pixels.
- The image uses the same off-white, blue, ink, and teal visual language as the application.

14 / INSTALLATION AND OPERATING COMMANDS

Installation and operating commands

Prerequisites

- Git installed and available in the terminal.
- Node.js 22.13.0 or newer.
- npm included with Node.js.
- Internet access for the first dependency installation and live public GitHub analysis.

First-time development setup

```
git clone https://github.com/krazyDarshan/logic_loop.git
cd logic_loop
npm ci
npm run dev

# Open http://localhost:3000
```

Production-mode local run

```
npm ci
npm run build
npm run start -- --port 3000

# Open http://localhost:3000
```

Updating an existing clone

```
git pull origin main
npm ci
npm run dev
```

Quality commands

Command	Purpose
<code>npm run lint</code>	Run ESLint against the source
<code>npm run build</code>	Create the Vinext production bundle in dist
<code>npm test</code>	Build, server-render through the Worker, and run product assertions
<code>npm run start -- --port 3000</code>	Serve the built bundle through Wrangler
<code>npm run db:generate</code>	Generate migrations if tables are later added
Common setup issue Do not run <code>npm run start</code> before <code>npm run build</code> on a fresh clone because dist is intentionally not stored in Git. If vinext or wrangler is not found, run <code>npm ci</code> first.	

15 / TESTING AND DEPLOYMENT

Testing and deployment

Verification completed

Check	Result
ESLint	Passed after replacing explicit any GitHub response types
Vinext production build	Passed all five environments: client references, server references, RSC, client, and SSR
Server-render test	HTTP 200 HTML with SkillNova metadata and all primary workspace labels
Feature-source test	Confirmed GitHub endpoints, PDF.js, Web Worker, jsPDF, and all three scoring functions
HTTP smoke test	Page, compiled CSS, and Open Graph PNG returned HTTP 200 in local Wrangler production mode
Fresh-clone build	GitHub commit 0fc069d installed with npm ci and built successfully
Deployment	Sites production deployment completed successfully

Deployment architecture

- npm run build creates dist/server/index.js plus static assets.
- The Sites Vite plugin copies .openai/hosting.json and optional Drizzle migrations into dist/.openai.
- worker/index.ts provides the Cloudflare-compatible request handler and image optimization route.
- The exact validated source commit and build archive were saved as a Sites version and deployed privately.
- Current URL: <https://skillnova-talent-intelligence.darshanbawaskar.chatgpt.site>

Dependency health

Audit follow-up required

The current npm audit reports 23 dependency findings: 4 moderate and 19 high. The application builds and runs, but these should be reviewed before a public production launch. Avoid npm audit fix --force without testing because it may introduce breaking upgrades.

16 / CURRENT LIMITATIONS AND PRODUCTION RISKS

Current limitations and production risks

Area	Current limitation	Production upgrade
Persistence	Candidate additions, shortlists, interviews, and safeguards reset on refresh	Add D1/PostgreSQL persistence and API routes
GitHub API	Unauthenticated rate limits and recent-event windows	Use a server-side GitHub App/token and cache normalized evidence
Resume parsing	Text PDFs only; no OCR, layout understanding, or robust entity extraction	Add OCR and structured document extraction
AI layer	No LLM, embeddings, LangGraph, or trained prediction model	Add model-assisted summaries with schemas and retain deterministic scores
Job semantics	Catalog and keyword overlap can miss related concepts	Add embeddings and pgvector with explainable feature blending
Code sandbox	Browser Worker eval is not a production security boundary	Use Judge0 or hardened isolated containers
Fraud detection	Identity and duplicate inputs are seeded; toggles are not enforcement	Implement certificate validation, profile similarity, and immutable audits
Authentication	Auth helper exists but the page does not consume it	Require authenticated recruiter/candidate roles
Bias testing	Protected traits are excluded by design but no formal fairness evaluation exists	Add slice-based monitoring, human review, appeals, and governance documentation
Hackathon import	Ranking uses seeded profiles rather than an organizer integration	Build event, team, project, presentation, and contribution ingestion

Hiring safety principle

Talent Score, match score, authenticity, and interview score should prioritize review rather than automatically reject or hire a person. Recruiters must inspect evidence, validate missing skills, and document the final human decision.

17 / RECOMMENDED ROADMAP

Recommended roadmap

Phase	Indicative effort	Deliverables
Phase 1 - stabilize	1-2 weeks	Resolve dependency findings, add error boundaries, authenticated GitHub access, better loading states, and end-to-end browser tests
Phase 2 - persist	2-3 weeks	Candidate/job/interview schemas, D1 or PostgreSQL storage, recruiter roles, audit records, and saved shortlists
Phase 3 - enrich	2-4 weeks	OCR, structured resume entities, repository README analysis, embeddings, and evidence cache
Phase 4 - verify	3-5 weeks	Judge0 code execution, question banks, anti-cheating controls, certificate verification, and duplicate detection
Phase 5 - expand	3-5 weeks	Hackathon ingestion, team contribution analytics, PPT/PDF pitch analysis, and organizer dashboards
Phase 6 - govern	Continuous	Fairness testing, model and formula versioning, appeals, retention rules, consent, monitoring, and security review

Recommended backend shape

- Keep the current Next.js interface and shared TypeScript score explanations.
- Move GitHub access, resume storage, and report persistence behind authenticated API routes or FastAPI.
- Use PostgreSQL with pgvector for durable profiles and semantic retrieval, or D1 for a simpler Cloudflare-native path.
- Use an LLM only for structured summaries, question generation, and feedback; do not replace the published scoring formulas with unconstrained output.
- Version every formula and store its input snapshot so a recruiter can reproduce a historical decision.

Best next investment

Persistence plus authenticated evidence ingestion will create more product value than adding another dashboard. It turns the current strong demonstration into a multi-user recruitment workflow.

18 / DEMO AND HANDOFF GUIDE

Demo and handoff guide

Five-minute demo sequence

Time	Action	Talking point
0:00-0:45	Open Talent Intelligence	Explain proof over paperwork and show the published Talent Score formula
0:45-1:30	Upload a resume and enter GitHub	Run live analysis and point to repositories, skills, confidence, and evidence sources
1:30-2:15	Open Job Matching	Edit one required skill and show rankings, gaps, and the percentage change immediately
2:15-3:10	Open Skill Verification	Submit a structured answer, reveal rubric components, and run the three code tests
3:10-3:50	Open Hackathon Hiring	Select a finalist and connect innovation, technical depth, and authenticity
3:50-4:35	Open Recruiter Workspace	Search in natural language, compare candidates, shortlist, and export a report
4:35-5:00	Open Trust Center	End with human approval, protected-trait exclusion, limitations, and the production roadmap

Questions the team should be ready to answer

- Why use transparent formulas? Because hiring scores require traceability and tuning.
- Is this using GPT today? No. The current deployed intelligence is deterministic; an LLM is a planned enrichment layer.
- Are GitHub counts perfectly complete? No. Public events are recent signals and the UI should describe them that way.
- Can the app reject candidates automatically? No. It is decision support with human approval.
- What is the strongest technical demonstration? Live evidence ingestion flowing through score, match, verification, recruiter comparison, and trust views.
- What would you build next? Persistence, authenticated ingestion, secure code execution, and real fraud checks.

Project handoff checklist

- Clone with npm ci and verify Node 22.13 or newer.
- Run npm test and npm run lint before merging changes.
- Do not commit node_modules, dist, .vinext, .wrangler, secrets, or environment files.
- Keep score formulas, labels, and report explanations synchronized.
- Clearly label sample, live, estimated, and verified evidence.
- Update this report after any architecture, formula, or deployment change.

End of report

SkillNova now functions as a coherent, deployable hackathon MVP with real evidence ingestion and reproducible calculations. Its strongest quality is not the number of screens; it is the connected path from evidence to an explainable recruiter decision. Production URL: <https://skillnova-talent-intelligence.darshanbawaskar.chatgpt.site>